Second Thought

A checkout that sometimes says not yet.

PROJECT DOSSIER · RESEARCH TO WORKING BUILD

PayPal AI Hackathon 2026 · Devpost

Submission deadline · Thu 12 Nov 2026, 12:00 pm PT

Prepared 5 November 2026

Everything measured here was run. Where something is claimed and not proven, this document says so — **including the parts nobody would have checked**.

CONTENTS

What is in this dossier

I **The research** — the contest, the field, the pain, and the idea those three produced

1. The contest as it is actually judged · 2. The field of 549 · 3. The pain, sourced · 4. The idea and its rules

II **The build** — what was constructed, and where each guarantee lives

5. Architecture · 6. The PayPal integration · 7. The AI boundary · 8. The security model · 9. The decision engine · 10. The screen

III **The evidence** — what is verified, what is not, and how it is checked

11. Verified against simulated · 12. Five things that broke · 13. Gates V0–V8 · 14. The scorecard

IV **The submission** — the words, the pictures, the three minutes

15. Devpost copy · 16. Assets · 17. The video

V **The honest parts** — what is missing, what would lose, what comes next

18. Limits · 19. The eight days · 20. Appendices

How to read the status marks

MARK	MEANING
PASS	Executed in this workspace, output recorded, reproducible by a command in Appendix C.
READY	Built and tested against a simulated transport that implements the same protocol. It has not spoken to PayPal or a model yet — one command and your credentials away.
OPEN	Not done. Named, sized, and owned.

> **One honesty note, up front**
>
> The live sandbox calls require credentials, and this workspace does not hold them. Every number in Part III that says **PASS** is a real measurement made here. Every number that says **READY** is a fact about code, not about your PayPal account. On your machine, with your `.env`, the two commands in §11 turn READY into PASS and write the artifacts a judge can look up in the dashboard. This dossier will be re-issued with those artifacts attached — until then it describes them accurately, as things that will happen rather than things that did.

The ten numbers

170 TESTS, 0.7 S	3 VERDICTS	5 PAYPAL CALLS	0 DEPENDENCIES	22 SECURITY CHECKS
549 REPOS SCANNED	3 IN OUR CLUSTER	47% PAID LATE	299 LINE CAP, PER FILE	0 USERS TESTED

What the contest rewards, and who else is in it

Six weeks of reading condensed into four pages: the rules that decide everything, the field that sets the bar, the pain that justifies the product, and the idea those three produced.

1 · The contest as it is actually judged

Two stages. **Stage 1 is pass/fail** and asks only two things: does the project fit the theme, and does it reasonably apply the required APIs. A beautiful project with a decorative PayPal button dies here and is never scored.

Stage 2 scores five criteria, equally weighted. The tiebreak goes to the first one listed, which is Technological Implementation — so in a field this size, where scores cluster, depth beats polish when the two compete for the same hour.

CRITERION	WHAT IT ACTUALLY ASKS	WHAT WE DID ABOUT IT
Technological Implementation	Is the PayPal and AI use thorough, skilful and verifiable — or a template with a logo?	Server-side OAuth, Orders v2, Invoicing v2, verified webhooks, idempotency, 170 tests
Design	A complete, coherent product, not a technical proof of concept	One page, three answers as sentences, empty and error states, no external assets
Potential Impact	A specific problem, a real audience, and a demo that shows the solution working for them	Sourced pain, a named shopper, and a refusal that runs rather than a slide that says "we care"
Innovation	Novel, and not the same as the other entries	3 entries in our cluster of 549; the inversion is the demo
Presentation	Does the video show it working end to end and explain who it is for	Beat sheet in §17, recorded after the outside-eye pass

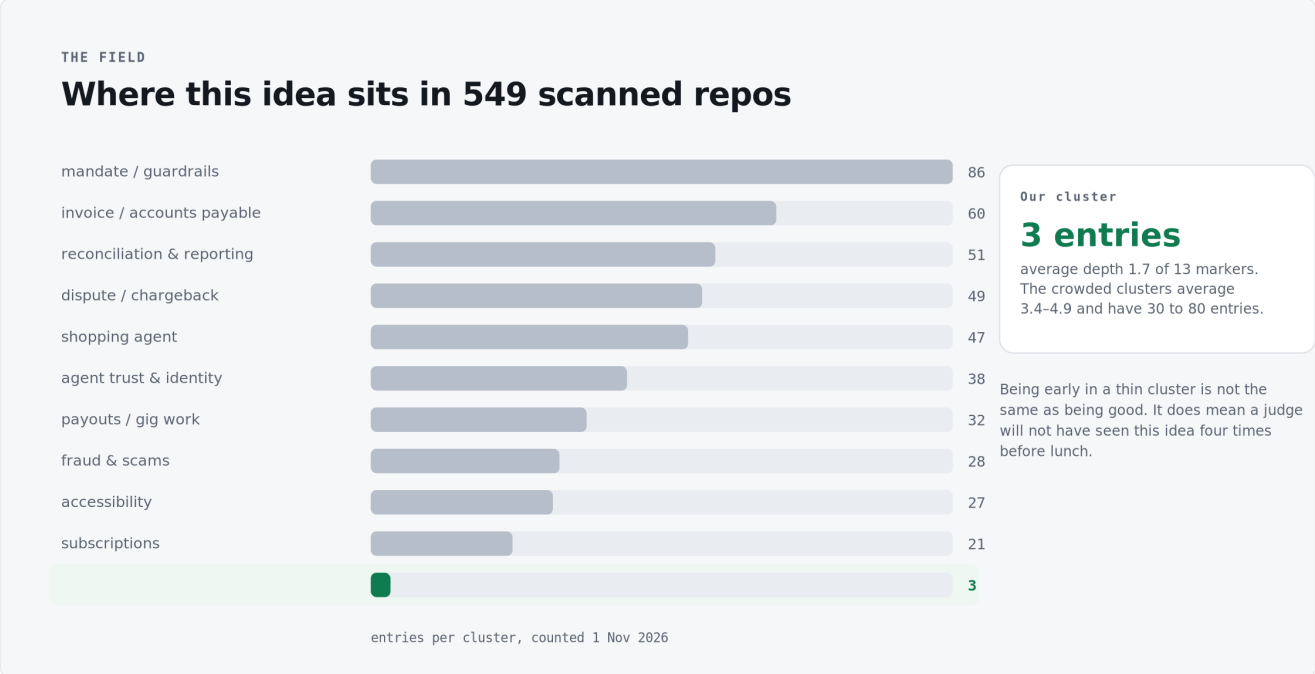
Dates and prizes

ITEM	DETAIL
Submission closes	**Thu 12 Nov 2026, 12:00 pm PT** = 13 Nov, 01:30 IST
Judging	1 Dec 2026 → 15 Dec 2026, 08:00 PT
Winners	on or around 21 Dec 2026
Prize pool	$67,500 cash. 1st $12,000 · 2nd $8,000 · 3rd $5,000
Honourable mentions	five × $5,000 — Most Creative, Most Impactful, Best Demo Delivery, Best Use of PayPal + AI, Best Use of Agentic Commerce
Registered	26,708 participants as of 2 Nov (reported). Registrations, not submissions — expect hundreds to ~2,000 entries. An estimate, labelled as one.

> **Two things to re-check before submitting**
>
> **The deadline hour.** The official rules say 12:00 pm PT; at least two third-party summaries say 2:00 pm PT. Plan for noon and confirm on the live listing. **The video must be public and under 3:00** — judges are not required to watch past it, and README, demo link and video all lock at close.

2 · The field of 549

Every public repo on the hackathon's topic list was scanned and clustered by what it does. The point was not to find competitors to copy, but to find the clusters that are already crowded and the ones that are empty.



Cluster density, 1 Nov 2026. Bar length is entries in that cluster.

MEASUREMENT	VALUE	WHY IT MATTERS
Public repos scanned	549	the sample the clusters are drawn from
With a live demo URL	230 (42%)	a deployed link is table stakes, not a differentiator
No detectable licence	95 (17%)	an eligibility item a sixth of the field fails
Scoring 10+ of 13 depth markers	7 (1.3%)	the actual bar for a prize, not the median
Credit / BNPL entries	**3**	average depth 1.7 — the thinnest cluster with a real problem behind it

What the top decile already does, and that we must match: a CI badge and a passing suite advertised in the README, a live demo with no login and no keys, a `/proof` page listing every measured number, a threat model, a red-team screen showing attacks blocked, an explicit statement that the demo stays online until 15 Dec, and a 2:18–2:37 demo video recorded weeks early.

3 · The pain, sourced

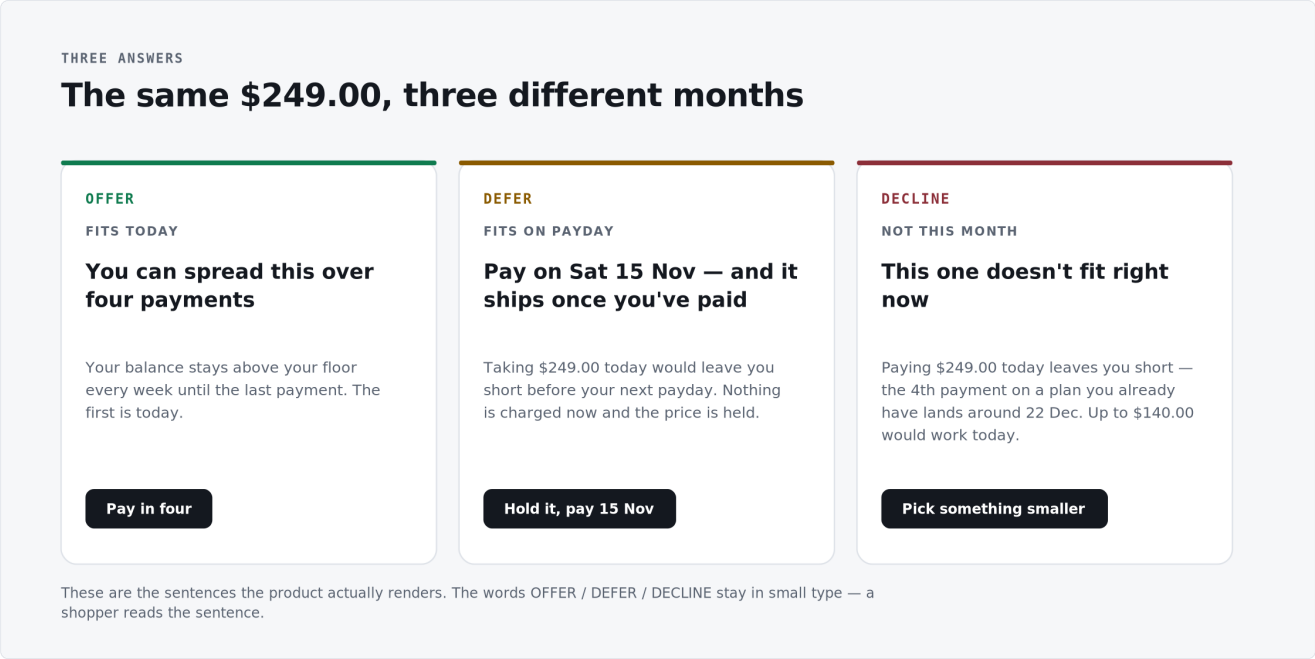
The product thesis is narrow: **the decision to offer a buy-now-pay-later plan is made without any model of whether this person can carry it.** The harm numbers are not about the product existing. They are about it being offered indiscriminately, in small repeated badly-timed payments.

FINDING	FIGURE	SOURCE
BNPL users who paid late at least once in the past year	**47%**	LendingTree, 2026 — up from 41% (2025) and 34% (2024)
BNPL users holding multiple simultaneous loans	**63%**	CFPB, January 2025
...of whom, across multiple lenders	**33%**	CFPB, January 2025
BNPL complaints, 2026 to date	**10,884**	CFPB complaint database — against 7,412 in all of 2025, +47% with two months to run

> **The caveat that belongs in the pitch, not the footnotes**
>
> The 47% is self-reported. The CFPB's dollar-weighted charge-off rate stays **under 2%**. Both are true and they measure different things. Saying "47% of BNPL users default" would be wrong, and a judge may know it. The correct sentence: "47% report paying late at least once, while dollar-weighted losses stay under 2% — the harm is concentrated in small, repeated, badly-timed payments, which is exactly the pattern a per-purchase check could prevent."

4 · The idea and its rules

Second Thought. At the moment someone is about to split a payment into four, ask a question nobody else asks: is this good for this person? Project their next 45 days, then answer with one of three things.



The three answers, in the exact words the product renders.

ANSWER	RULE	WHAT ACTUALLY HAPPENS
OFFER	every one of the four payments leaves the balance at or above the floor, every day for 45 days	PayPal `AUTHORIZE` hold, captured on the approved date
DEFER	that fails, but a single payment dated to the next payday passes, and the payday is inside the 21-day deferral window	An Invoicing invoice due on payday. **The merchant ships when it clears**
DECLINE	neither works, including the partial ladder	Nothing offered. The collision is named and the largest amount that would work is quoted

The arithmetic, which is deliberately boring

- **Horizon:** 45 days. Long enough to catch next month's rent, short enough to stay honest.
- **Floor:** `max(10% of monthly income, $50)`. Not a credit score, not a limit — a "don't let this week break you" line.
- **Instalments:** four, fourteen days apart.
- **Deferral window:** 21 days. Beyond that it is not a deferral, it is a different month.
- **Determinism:** the same calendar and the same amount always produce the same answer. There is no sampling, no temperature, no model in this path.
- **Partial ladder:** 90/75/60/50/40/25/10% now, remainder on payday — a fixed ladder, because feasibility is not monotonic in the split and a binary search would be unsound.

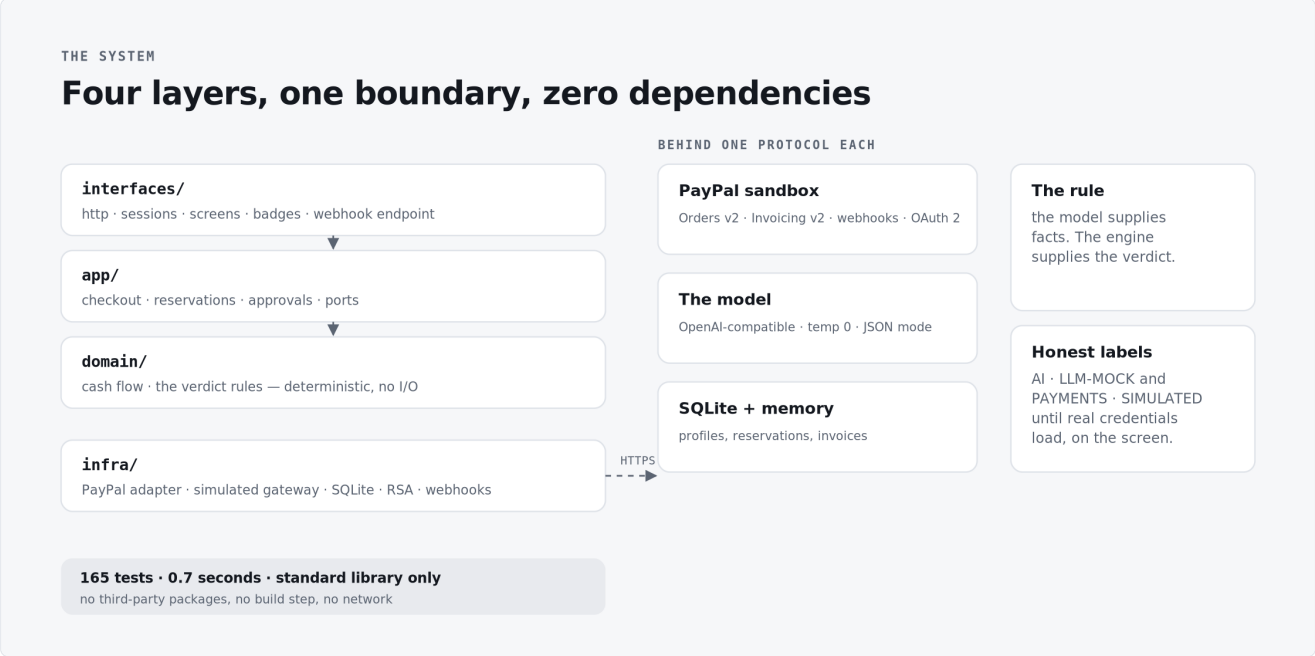
> **Why this is not another BNPL app**
>
> **DEFER is pay-before-ship.** The merchant extends no credit and funds no receivable; the shopper never borrows; the goods move when the money arrives. Compare a card decline, where a good customer is simply lost. Here the sale moves to a date the shopper can actually meet. The trade is timing, not volume — and it is stated that way, because "volume preserved" is a claim we cannot support.

What could make this derivative, and the escape route

RISK	MITIGATION, APPLIED IN THE BUILD
"Responsible lending" reads as a lecture	The refusal carries a number: "Up to $140.00 would work today." Show the maths and the option, never "you can't afford this"
A judge asks why PayPal would reduce its own BNPL volume	Because a denied-then-explained shopper returns and a silently-blacklisted one churns — and because the alternative to a deferral is a lost sale, not a full-price one
Looks like a budgeting app	It has no dashboard, no categories, no goals. It has one question and three answers, inside a checkout

What was constructed, and where each guarantee lives

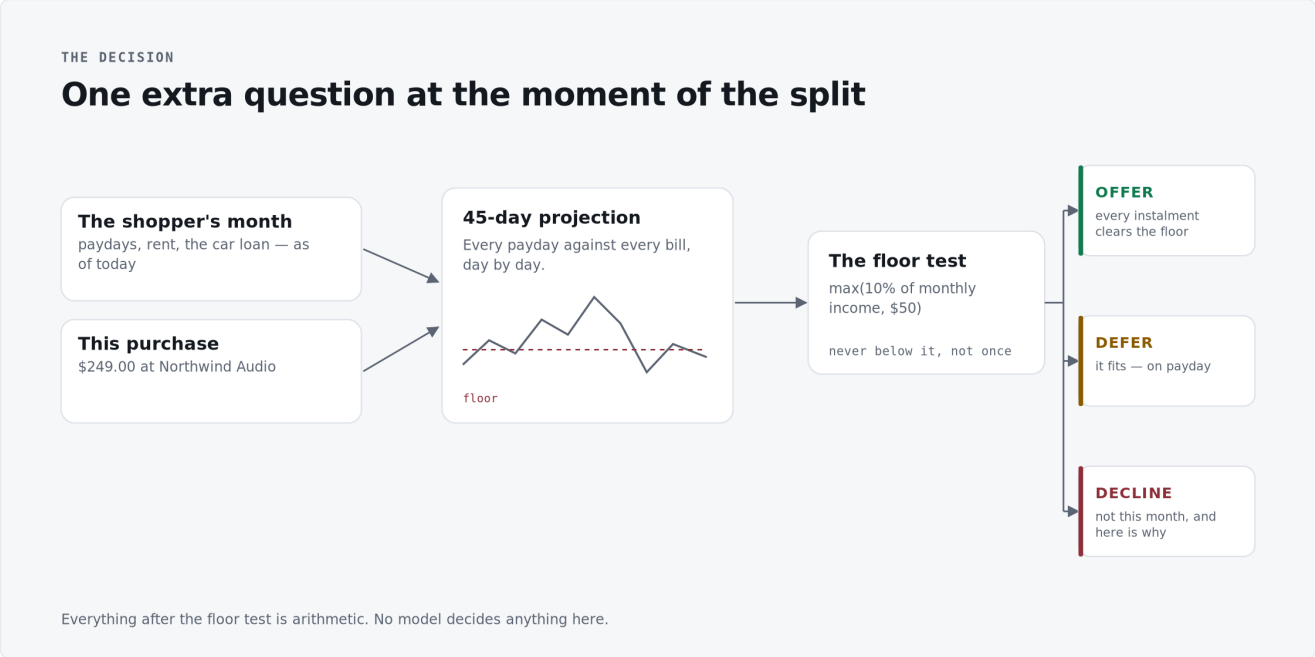
Seven and a half thousand lines of Python, no third-party packages, every file under 300 lines. The layering is not decoration: it is what makes the AI boundary and the security model checkable by a person in ten minutes.



Four layers. `domain/` decides and does no I/O; `infra/` is the only layer that talks to anything.

5 · Architecture

LAYER	RESPONSIBILITY	FILES
`domain/`	Cash flow, the floor test, the three verdicts, the safe-amount search. Pure functions over dataclasses.	`cashflow.py`, `engine.py`, `reservation.py`
`app/`	Orchestration: quote → propose → approve → execute. The reservation lifecycle. The approval gate.	`checkout.py`, `approvals.py`, `ports.py`
`ai/`	Reading unstructured text into dated amounts; the template fallback; the agent that fuses them and refuses bad input.	`extract.py`, `template.py`, `agent.py`
`infra/`	PayPal adapter (live), simulated gateway, SQLite, RSA, webhook verification, webhook idempotency.	`paypal_live.py`, `gateway.py`, `db.py`, `rsa.py`, `webhook.py`
`interfaces/`	HTTP, sessions, badges, the shopper screen, the plain-language layer, the webhook endpoint.	`http.py`, `session.py`, `screens.py`, `badge.py`, `personas.py`

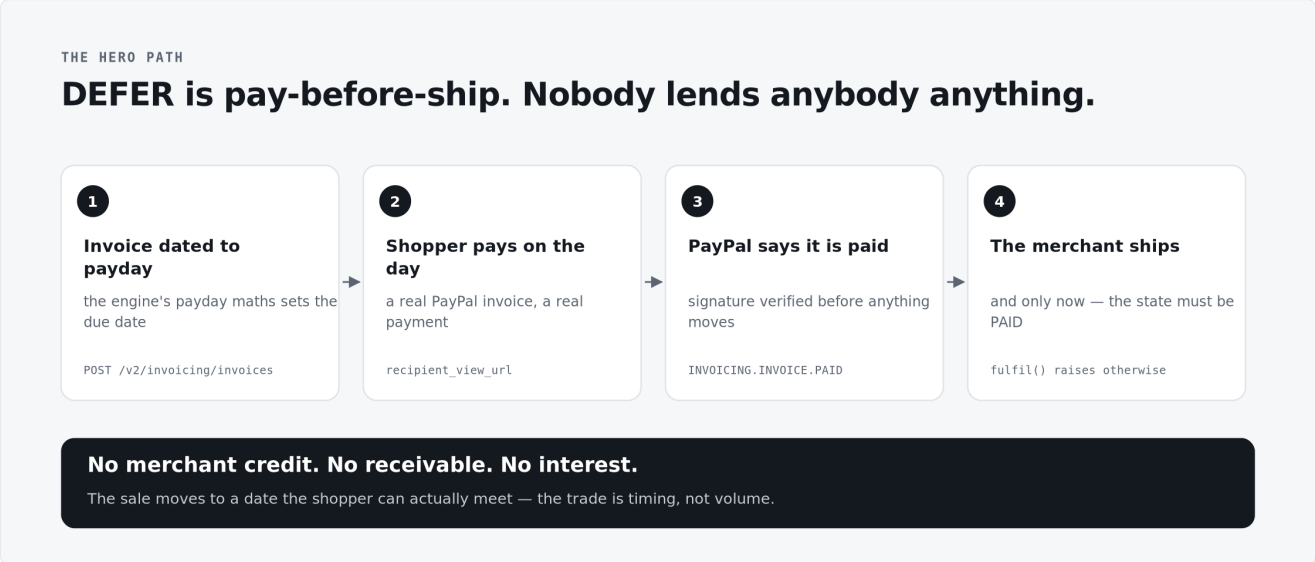


The path a single purchase takes. Everything after the floor test is arithmetic.

6 · The PayPal integration

Every call is server-side. No client secret reaches the browser, and the live adapter refuses to run if `PAYPAL_ENV` is anything but `sandbox`.

PAYPAL SURFACE	ENDPOINT	USED FOR
OAuth 2 client credentials	`POST /v1/oauth2/token`	every call below; token cached for its 9-hour life
Orders v2 — create + AUTHORIZE	`POST /v2/checkout/orders`	an OFFER reserves the funds rather than charging them
Capture	`POST /v2/payments/authorizations/{id}/capture`	on the approved date
Void	`POST /v2/payments/authorizations/{id}/void`	release when the shopper changes their mind
Invoicing v2	`POST /v2/invoicing/invoices`	the DEFER invoice, `payment_term.due_date` set by the engine's payday maths
Webhooks	`/api/webhook`	signature-verified; `INVOICING.INVOICE.PAID` is what turns a deferral into a shipment
Idempotency	`PayPal-Request-Id`	a retried call returns the first result instead of creating a second charge



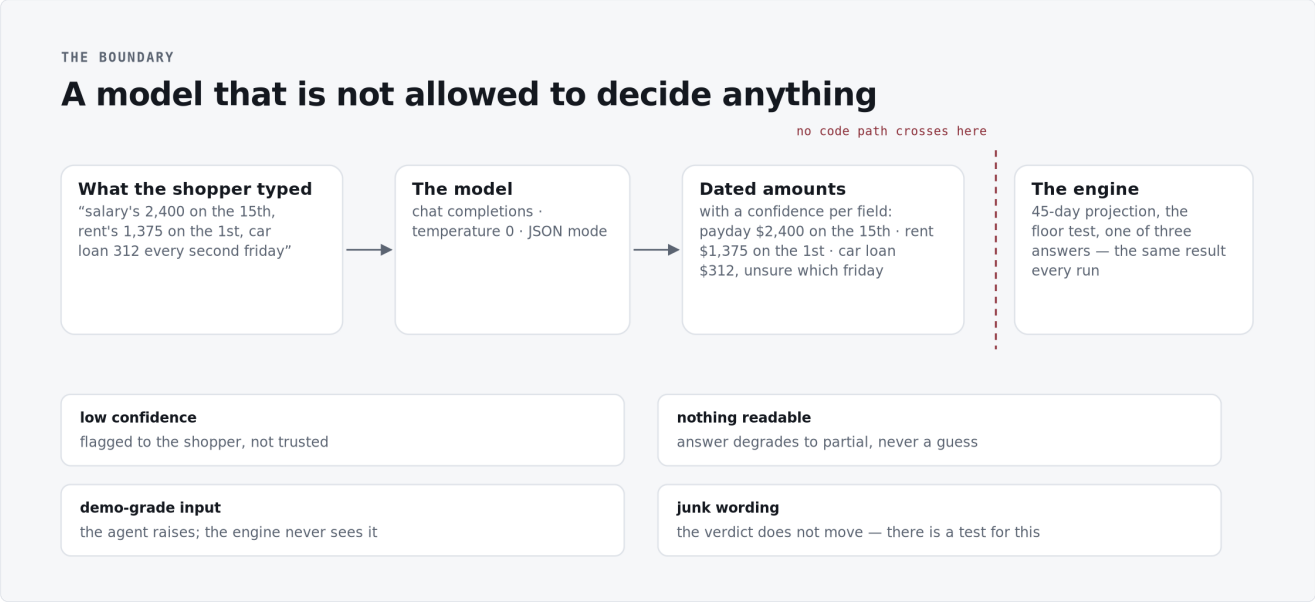
The hero path. Four steps, three of them PayPal's.

Webhook verification, in full

The signed string is `transmissionId|transmissionTime|webhookId|crc32(rawBody)` computed over the **raw bytes**, checked with RSA-SHA256 PKCS#1 v1.5 against a certificate fetched from a URL inside the event. The certificate URL must be HTTPS and on a PayPal host, and that check lives in the key-fetch path, not only in the fetch helper — a security check in one code path is not a check. The implementation uses only the standard library: no `cryptography`, no `requests`.

7 · The AI boundary

The model's only job is reading a human sentence into structured facts. It cannot decide anything, and that is enforced structurally rather than promised in a README.



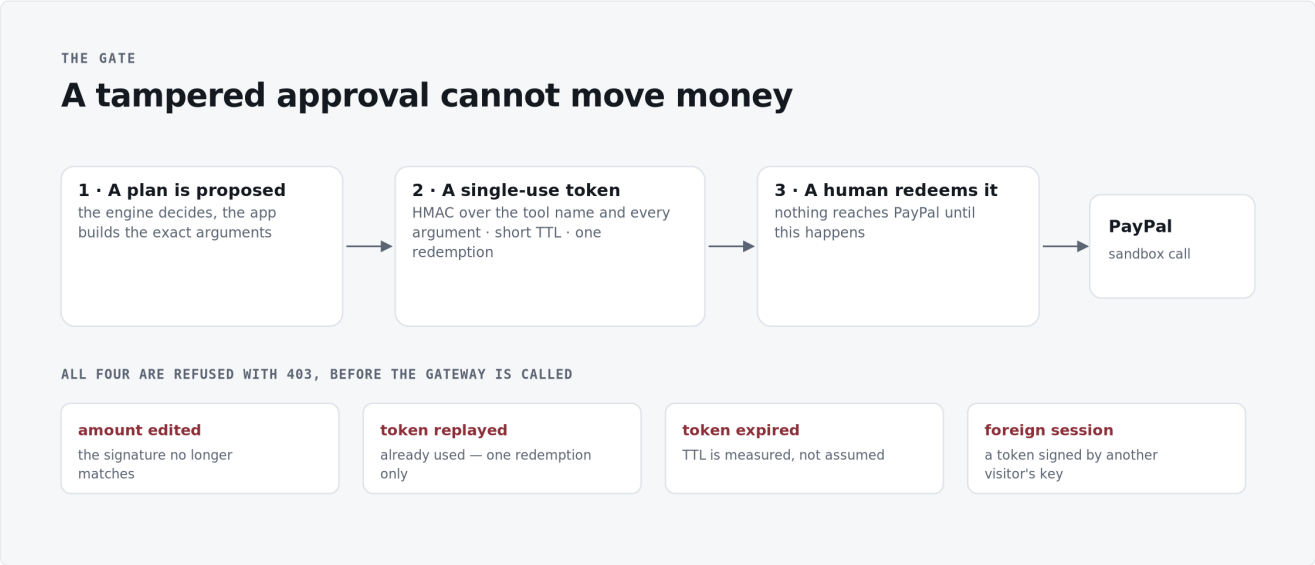
The wall. `Agent.profile_from` raises rather than pass a low-grade extraction through.

FAILURE MODE	WHAT THE SYSTEM DOES
Low confidence on a field	The field is kept, capped in confidence, and **flagged to the shopper** with a question. It is not silently trusted.

Nothing readable	The answer degrades to `partial` or `demo`. It never guesses and never crashes.
Extraction graded `demo`	`profile_from` raises. The engine never sees it.
Junk wording, valid facts	The verdict does not move. There is a test for exactly this.
No key configured	A deterministic template reader runs instead, and the on-screen badge says `AI · LLM-MOCK`.

> **Why the boring choice is the good one**
>
> Most submissions put the model in charge and hope. Here the model is a very good parser and nothing else: `temperature: 0`, JSON mode, per-field confidence, and an agent that raises on anything it cannot stand behind. It is the least fashionable use of an LLM in this competition and the only one we can defend line by line.

8 · The security model



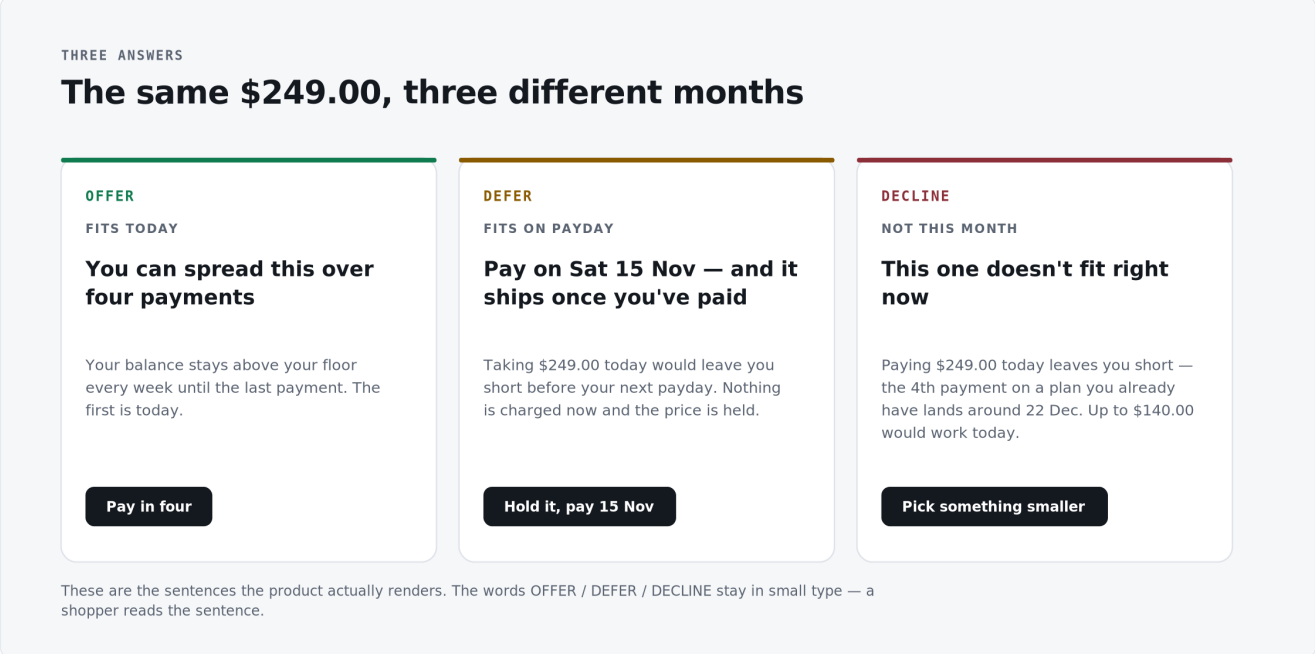
Money moves only through a token that is bound to one action, one amount, one session.

CONTROL	IMPLEMENTATION
Session isolation	A session per cookie, in memory, with its **own signing secret**. A token minted in one visitor's gate cannot verify in another's — this was a real defect, found and fixed.
Human approval	HMAC over the tool name and the exact arguments, single use, 600-second TTL. Nothing reaches PayPal until it is redeemed.
Tamper refusal	An edited amount, an altered due date, a replayed token, an expired token and a foreign token are all refused with **403 before the gateway is called**.
Rate limits	Two-key: 30 model calls per session and 90 per IP per 600 s, so clearing a cookie does not buy a fresh budget.

Spend cap	In-process daily call budget (1,500) as a backstop; the provider-side cap is the real protection and is set in the account.
Health endpoint	`/healthz` returns four booleans and a status word. No keys, no paths, no versions.
Diagnostics	`/diagnostics` returns 404 unless a token is configured, and counts only when it is.

9 · The decision engine

The core is a projection over a horizon, run once per candidate schedule. Its honesty comes from what it refuses to do: it will not spread a balance thinly and call it affordable, and it will not offer an instrument whose date is already in the past.



Three months, the same $249.00. The refusal quotes the amount that would fit.

10 · The screen

One page. Three months visible without a click, each with its amount. The answer arrives as a sentence, not a status code, and the internal vocabulary never reaches the shopper — a test fails the build if words like verdict, collision or token appear in a sentence a shopper reads.

ELEMENT	DECISION
Badges	`AI · LLM` / `AI · LLM-MOCK` and `PAYMENTS · SANDBOX` / `PAYMENTS · SIMULATED`, computed from configuration and never overridable by argument
Assets	Inline CSS, inline SVG favicon, no external fonts, no CDN, no analytics — it renders on a locked-down laptop with no network
Empty state	"Pick one of the three and the answer appears here" — the page is never a blank void
Free text	Reads what you typed, shows what it understood, flags what it is unsure of, and decides nothing from it
Mobile	Viewport meta, fluid cards, tap targets above 44 px

What is verified, what is not, and how to check both

A claim is worth what the check behind it is worth. This part separates the measurements from the intentions, and lists the five things that broke along the way — because that list is the most convincing page in the document.

11 · Verified against READY

CAPABILITY	STATUS	EVIDENCE
Cash-flow engine and verdict rules	PASS	deterministic, unit-tested, three verdicts reproduced on any date for sixty days forward
Test suite	PASS	170 tests, 0.7 s, no network, stdlib only
Session isolation & rate limits	PASS	22 checks over a live socket: two visitors, distinct keys, 200 → 429 at call 31, one visitor unaffected
Approval tampering	PASS	unknown, forged, foreign, amount+1, altered-date, replayed and expired tokens all 403
Webhook signature verification	PASS	RSA-SHA256 over the canonical string, verified on a live socket; missing header → HTTP 400 with the field named
Deploy configuration	PASS	`preflight_deploy.py` 17/17; mutation test 14 caught / 3 survived by design
Five live PayPal calls	READY	written, tested against a simulated gateway with the same protocol; exit 2 without credentials
Live model extraction	READY	OpenAI-compatible client written and unit-tested against a fake transport
Public URL	OPEN	not deployed; `render.yaml` is preflighted and ready
Video	OPEN	beat sheet written, nothing recorded
Human validation	OPEN	n = 0. Conversations scheduled, not done

The two commands that turn READY into PASS

```
python3 demo.py --live       # V3: OAuth → AUTHORIZE → capture, → void, → INV2 invoice, → same Request-Id twi
python3 demo_ai.py --live    # V4: badge flips to AI · LLM, three messy passages, tampered approval refused
```

Each writes `artifacts/v3/<timestamp>/` and `artifacts/v4/<timestamp>/` containing every raw response and, for each id, the dashboard location where a judge can find it. Terminal output is not evidence; the dashboard is.

12 · Five things that broke

1 · The demo broke by itself, and that was the best bug

The three demo months were pinned to November 2026 dates, and `/api/plan` passed a hardcoded `date(2026, 11, 1)`. Nothing in the code changed; the calendar moved. Ben's payday passed, the reservation refused a date in the past, and the DEFER path — the hero of the pitch — returned **HTTP 500**. Judging runs 1–15 December, when every date would be stale and a demo the rules require to be runnable would not run.

Fix: months generated relative to today; a test walks sixty days forward asserting all three verdicts still appear; a lint fails the build if a literal date re-enters the demo path.

2 · The .env nothing read

`.env.example` said "copy to `.env` and fill in", and only the precondition checker had a reader. Anyone following the instructions would set up credentials and then be told they were missing — while looking straight at them.

Fix: `config.py` loads `product/.env` at import, real environment variables win, and `ST_ENV=production` ignores the file entirely so a stray `.env` can never override a dashboard.

3 · `pay_on` was populated for OFFER

The recommended option for an OFFER carries an `invoice_due` — the first instalment — and the quote copied it into `pay_on`. The screen would have told a shopper who can pay today to wait.

Fix: the field is populated only for DEFER, with a test asserting it.

4 · Dollars against minor units

A hundred-fold error waiting to happen. Every amount in the system is an integer of cents, and there is exactly one place that converts. The other half was payday arithmetic: a fortnightly cadence with no anchor date must not borrow today's date at high confidence, so it caps the confidence and asks a question instead.

5 · A security check in one code path

The webhook certificate URL was validated in the fetch helper and not in the key path, so a second caller could reach an unvalidated URL. One check in one place is not a check.

Fix: validation moved into `_key()`, where every caller passes through.

13 · Gates V0–V8

GATE	STATUS	WHAT IT ASKS, AND WHERE IT STANDS
V0	OPEN	Preconditions: 14 environment variables proven real. Locally: credentials absent; on your machine, satisfied.
V1	READY	Pay Later sandbox probe — confirm the band from the response, never assume. Probe page built.
V2	PASS	Code correctness: 170 tests, preflight 17/17, mutation 14/3, hand-read assertions.
V3	READY	Five live calls, each independently verifiable in the dashboard. Instrument complete; credentials absent here.
V4	READY	Live AI: safety half proven offline, quality half one key away.

V5	PASS	Isolation, limits, no unauthenticated money action — 22 live checks. One item pending a redeploy.
V6	OPEN	Deployment and durability. Item 6 (calendar for 1 and 10 Dec) done; the rest needs the URL.
V7	CLOSED	Human validation, **n = 0**. Capture tooling and a rule engine are built; the conversations are the owner's.
V8	READY	Outside-eye rehearsal: screen, protocol and timer built; **n = 0** until a stranger uses it.

14 · The scorecard

Scored against the five judged criteria, with the anchors published so a number means something. `scripts/scorecard.py` regenerates this from the repo.

CRITERION	NOW	AFTER THE PLAN	WHY NOW IS WHAT IT IS
Technological Implementation	1.7	4.5–5.0	depth is real, verifiability is zero: no PayPal call has happened
Design	4.0	5.0	the screen is product-shaped; nobody can open it
Potential Impact	4.1	4.5–5.0	sourced pain and a working refusal; no human has used it
Innovation	4.5	4.5–5.0	3 entries in a field of 549; self-scored, so discounted
Presentation	0.0	4.5	nothing recorded — the largest recoverable loss
Mean	**≈ 3.0**	**≈ 4.5**	everything weak is weak for one reason: nothing has left this machine

Points per hour, ranked

MOVE	HOURS	Δ MEAN
`demo.py --live` — five real calls	2	+0.4
Deploy + licence + `git init` + push	1	+0.3
`demo_ai.py --live` — real model behind the badge	1	+0.2
Webhook id + one real transmission	0.5	+0.1
`/proof` page and `SECURITY.md`	2	+0.2
Video, 2:40	5	**+0.9**
V8 rehearsal, then fix the copy it exposes	1	+0.2
V7 conversations, or accept n = 0	8	+0.2

The three technical moves at the top are one afternoon. The video is worth more than everything technical that remains — and it is the one artefact every judge is guaranteed to consume.

The words, the pictures, the three minutes

15 · Devpost copy

FIELD	ANSWER
Project name	**Second Thought**
Elevator pitch	A checkout that says not yet. AI reads your month; if four payments would break it, PayPal invoices you on payday instead — and the merchant ships only once you've paid. (169 / 200 characters)
Built with	Python, PayPal, PayPal Orders API v2, PayPal Invoicing API, PayPal Webhooks, PayPal Sandbox, OAuth 2.0, SQLite, AI, Large Language Models, OpenAI API, JSON Mode, HTML, CSS, JavaScript, Render, GitHub, unittest
Sponsor tool	**Render** only. AG Grid, APIMatic, Postman, Elastic, Zapier, Channel3, Bryntum, Astropods, KERNEL: left blank rather than claimed.
PayPal experience	**8 / 10**, with the friction named: Pay Later eligibility is client-side, several v2 endpoints return 204 without `Prefer: return=representation`, OAuth answers form-encoded, invoices sit as DRAFT until sent, and webhook verification fails closed with no explanation

The story, in the order the form asks for

1. **Inspiration** — every BNPL checkout asks how much and which country, never "can you carry this". Opens on the sourced numbers, then immediately on the counter-argument to its own statistic.
2. **What it does** — the projection, the floor test, three verdicts, and the two screens quoted verbatim.
3. **How we built it** — four layers, the PayPal surface by name, the AI boundary, the security model, 170 tests.
4. **Challenges** — the sandbox cannot originate a real Pay Later plan; the demo broke by itself; money in minor units; webhook verification with no dependencies.
5. **Accomplishments** — pay-before-ship enforced in code; the refusal that carries a number; a model with no vote; and the admission that we have not tested with users yet.
6. **What we learned** — the payment is not the decision; deterministic money, fluent language; fake transports beat mocks; nobody reads status codes.
7. **What's next** — five conversations this month, a merchant view, closing the loop on a native Pay Later plan, twelve weeks of dates.

16 · Assets

Eighteen assets, SVG source and PNG, generated from one palette so the logo, the diagrams and the product page cannot drift apart. Placeholders are pointless here: the mark is a solid thought and an open one, the same moment looked at twice.

ASSET	USE

`logo-primary`, `logo-mono-white`, `logo-mark-white`	README header, video title and end cards, this cover
`app-icon` 512², `favicon` 256²	avatar, listing, browser tab (the page serves the favicon as an inline data URI)
`devpost-thumbnail` 1280×720	the only image most judges see before deciding to click
`og-card` 1200×630	repository social preview
7 diagrams	story illustrations and the video's technical beat

17 · The video

Two minutes forty, no slides, recorded after the outside-eye pass so the stuck points write the script rather than our enthusiasm.

TIME	ON SCREEN	WHY IT IS THERE
0:00	The DECLINE screen, unedited	Lead with the refusal. It is the whole idea in one sentence and no other entry opens with a "no"
0:12	47% paid late · 63% stacked · 10,884 complaints	Impact, with the counter-argument on screen for two seconds so it reads as honesty, not hedging
0:25	Cleo: collision named, "up to $140.00 would work today", then re-run at $140 → OFFER	The product proves itself on camera
0:55	Ben: DEFER → invoice dated to payday → paid → ships	The hero path, with the PayPal sandbox invoice visible in the dashboard
1:35	Free text: messy month → what it read → flagged field	AI, visibly doing something a regex cannot
1:55	Tampered approval → 403	Technological Implementation: a failure path, deliberately shown
2:15	The boundary diagram, 6 seconds	"the model supplies facts, the engine supplies the verdict"
2:30	Closing card; "researched and built by one person, not yet tested with users"	The claim we are allowed to make, made plainly

> **Rules for the recording**
>
> Under 3:00 with the runtime checked, not assumed. No music that needs a licence. No PayPal logo, wordmark or trade dress anywhere in frame — the word is fine in body copy. Every number spoken must exist in the repo. And the DECLINE case must be in it; a video that only shows an offer is a video of a different, much more common product.

What is missing, what would lose, and what happens next

18 · Limits, stated plainly

LIMIT	WHAT IT COSTS, AND WHAT WE SAY INSTEAD
No live PayPal call in this workspace	Stage 1 asks whether PayPal is genuinely applied. The adapter is written and tested against a protocol-identical simulation; it has not moved sandbox money here. On your machine it does — and the artifacts go in the appendix.
No users (n = 0)	The claim is **"researched, built and tested by us"**. Never "validated". Five conversations are scheduled; if they do not happen, the pitch says so early rather than being found out late.
No stranger has tried the screen (V8, n = 0)	We wrote it to be understood and have not watched anyone try. The plain-language audit is a proxy, and it caught two real defects — it is not the same as comprehension.
No deployment	Three gates and the video stand on a public URL. Until it exists, every Design claim is unverifiable by anyone.
Session state is in memory	A redeploy logs everyone out. Correct for a demo, and said out loud rather than discovered by a judge.
Pay Later cannot be originated in sandbox	Eligibility is decided client-side from amount and country. So the mechanic sits on AUTHORIZE and Invoicing, which we control end to end — a stronger promise than borrowing, and honestly labelled as a different primitive.

Three ways this loses anyway

1. **The safe path gets taken.** A lookalike "AI shopping agent with a budget" sits in a cluster of 47. Anything that reads as responsible-lending advice is scored as a lecture. The refusal must be shown as a feature with a number in it.
2. **The claim outruns the evidence.** "Validated with users" at n = 0 is the fastest way to lose a judge who checks. The honesty costs nothing and buys the rest of the document.
3. **It is not there in December.** The rules require availability until 15 Dec. Memory sessions, a sleeping free tier, or a stale demo data set each break that — the date bug in §12 was exactly this failure, and was found by luck rather than by design.

19 · The eight days

WHEN	WHAT
Day 1	`demo.py --live` and `demo_ai.py --live`. Collect five artifacts and three dashboard screenshots. Badge flips.
Day 2	`git init`, `.gitignore`, `LICENSE`, push, deploy to Render on Starter, add the webhook id, take one verified transmission.
Day 3	`/proof` page, `SECURITY.md`, README privacy line, screenshots. Scorecard run with `--url`.

Day 4	V8: one person, the URL, the repo, the README, no narration. Write down every stuck point — the script below is the whole of it.
Day 5–6	Video: record from the stuck points, 2:40, under 3:00, DECLINE included.
Day 7	Devpost form, thumbnail, all fields, internal submit and tag the commit.
Day 8	Buffer. Nothing new ships. `main` stays stable to 15 Dec.

> **The V8 rehearsal, as a script**
>
> Ninety seconds, one person who has never seen the screen, no explanation from you. Hand over a phone on `/checkout?persona=ada` and say nothing. Then ask these five questions in order and write down their answers verbatim:
>
#	ASK	PASS LOOKS LIKE
1	Read the first screen out loud.	No question about what the product is; no request for context.
2	Pick “defer”. What did you just agree to?	“Half now, half in a fortnight” without opening any help text.
3	You want to pay early. Where do you go?	Finds the change control in under twenty seconds.
4	Whose money is this, and is it a loan?	Says the shop’s money, through PayPal; says it is not credit.
5	Change your mind and go back. Dead end?	Returns cleanly; the badge still tells the truth about what is paid.
>
> The first sentence out of their mouth containing “wait” or “so…” is the copy you rewrite that night — not a new feature. n = 1 beats n = 0 by enough that it is worth the hour.

20 · Appendices

A · File map

PATH	WHAT IT IS
`product/secondthought/domain/`	the verdict engine — cash flow, floor test, three answers, safe amount
`product/secondthought/app/`	checkout orchestration, reservation lifecycle, approval gate
`product/secondthought/ai/`	model reader, template fallback, the agent and its refusals
`product/secondthought/infra/`	PayPal adapter, simulated gateway, SQLite, RSA, webhooks
`product/secondthought/interfaces/`	HTTP, sessions, badges, personas, the shopper screen
`product/tests/`	170 tests across 12 files
`product/scripts/`	preflight, V0 check, two-jar security suite, cold check, V7 capture, V8 observer, scorecard, asset generators
`product/assets/`	17 generated assets, SVG and PNG, plus a contact sheet

`docs/research/`	50 documents: contest, intent, field, pain, capability, concepts
`docs/plan/`	the plan of record, one document per gate, the scorecard, the submission kit

B · The environment contract (14 variables)

VARIABLE	PURPOSE
`PAYPAL_CLIENT_ID / _SECRET`	sandbox app credentials; the badge reads SANDBOX only when both are present
`PAYPAL_WEBHOOK_ID`	the id a webhook signature is verified against
`PAYPAL_ENV`	`sandbox`; the live adapter refuses anything else
`LLM_API_KEY / _MODEL / _BASE_URL`	any OpenAI-compatible endpoint; absent means the template fallback and an honest badge
`LLM_DAILY_CALL_BUDGET`	in-process backstop (1,500); the provider-side cap is the real protection
`APPROVAL_SECRET`	signs approval tokens; a shipped default is reported by `/diagnostics`, not hidden
`DIAG_TOKEN`	gates `/diagnostics`; unset means 404
`RENDER_API_KEY`	read only by the precondition check, to prove the instance is paid rather than free
`ST_ENV / ST_DB_PATH / PORT`	runtime; `ST_ENV=production` also disables `.env` loading

C · Commands

```
cd product

python3 -m secondthought.cli              # the whole engine in a terminal, no network
python3 scripts/serve.py                  # the page on :8000
python3 -m unittest discover -s tests -q  # 170 tests, 0.7 s
bash scripts/v5_two_jars.sh               # 22 live security checks
python3 scripts/preflight_deploy.py       # 17 checks on render.yaml
python3 scripts/v0_check.py               # the precondition gate
python3 demo.py --live                    # V3, five PayPal calls
python3 demo_ai.py --live                 # V4, the real model
python3 scripts/scorecard.py --url …      # the five criteria, measured
python3 scripts/v8_observer.py --precheck --url …
python3 scripts/make_assets.py && python3 scripts/make_diagrams.py \
  && python3 scripts/make_diagrams_tech.py && python3 scripts/make_asset_index.py
```

D · Sources

Official Rules and the Devpost listing, retrieved 1 Nov 2026 · LendingTree BNPL late-payment survey, 2026 · CFPB BNPL market report, January 2025 · CFPB consumer complaint database, BNPL keyword, 2025–2026 · PayPal developer documentation: Orders v2, Invoicing v2, webhooks and the certificate chain · the 549-repo field scan performed for this project, 1 Nov 2026. Every figure in this dossier traces to one of these; the ones that do not are labelled as estimates.



Closing

The strongest thing this project has is not the AI and not the API surface. It is that the system is willing to say no, and that the person who built it is willing to write down what did not work. Both are rare in a field of five hundred.

Second Thought · Python 3.11+ · no dependencies · 170 tests · sandbox only · one page, three answers, one honest badge.

Researched, built and measured between 1 and 12 November 2026. Twenty pages, eight figures, one open gate that matters. This document was typeset from `docs/DOSSIER.html` by `docs/tools/build_dossier.py` and rebuilt from source every time a number in it changed — which is the only reason to trust the numbers.